

CSCE 679 DATA VISUALIZATION · MODULE 07 · LECTURE NOTES

Networks and Hierarchies

Structure the plane does not naturally hold.

MODULE OUTCOMES

- Explain node-link layout and its limits.
- Explain the matrix alternative and its trade.
- Explain hierarchy layouts and what each preserves.
- Explain when a network should not be drawn at all.
- Choose a representation from the task.

1 Node-link

A FORCE-DIRECTED LAYOUT HAS NO PRINCIPLED READING, AND IT IS THE DEFAULT EVERYWHERE

The layout minimises a physical energy function that has no relationship to the data's meaning — springs on edges, repulsion between nodes — so **position encodes nothing in particular, which wastes the highest-ranked channel entirely** (Module 02 §1).

And it is **non-deterministic: two runs produce different pictures of the same graph**, which means **any apparent cluster may be an artefact of the random initialisation** rather than a feature of the data — and the reader cannot tell which.

Beyond a few hundred nodes it becomes a hairball — a dense region conveying only the fact that the graph is dense, which is a fact better stated as a number (§4).

But it is **genuinely good for small graphs where the topology is the point** — up to perhaps a hundred nodes, where **the reader can trace paths, see the neighbourhoods, and perceive the structure**, which is a real capability no other representation offers.

- **Few enough nodes to trace** — which is the binding constraint, and is roughly one to two hundred depending on the edge density.
- **Position encoding something, wherever you can arrange it.** A layered layout for a directed acyclic graph, geographic position for a spatial network, or a computed attribute on one axis — all of which are better than a force layout's nothing.
- **Minimal edge crossings, which is what most layout algorithms actually optimise** and is measurably helpful for path tracing, which is the task node-link is for.
- **And a deterministic layout**, so that **the picture is stable across runs and comparable across versions of the data** — which matters for anything that will be regenerated.

- Which together mean: use a force layout for exploration and something principled for publication. A figure that changes between runs cannot support a claim about structure, which is CSCE 676 Module 04 §3's stability argument arriving as a visualisation requirement.

2 The matrix

AN ADJACENCY MATRIX SCALES WHERE NODE-LINK DOES NOT, AND LOSES PATH-FOLLOWING

Rows and columns are the nodes; each cell encodes the presence or weight of an edge — so the layout is entirely deterministic, there is no occlusion whatsoever, and it scales to thousands of nodes while remaining legible.

And the row and column ordering is the design — a well-ordered matrix makes clusters appear as solid blocks along the diagonal, which is a readable and quantitative view of community structure (CSCE 676 Module 07 §3) and shows the between-cluster density too.

While an alphabetically ordered matrix shows nothing at all — the same data, unreadable — which is Module 03 §4's sorting point with unusually high stakes: the ordering is not a refinement here but the entire difference between a useful figure and noise.

But you cannot trace a path in a matrix — following a three-hop connection requires three separate lookups in different parts of the figure — which is exactly the task node-link does well and the matrix does not do at all.

The task	Use	Why
Trace a path between two nodes	Node-link	The matrix requires a separate lookup per hop (§2).
Find clusters	An ordered matrix	Blocks on the diagonal, read quantitatively rather than guessed from a layout.
Find the high-degree nodes	Either, or a bar chart	Degree is a value, and a sorted bar chart beats both network views (§4).
Judge the overall density	A matrix, or a number	A hairball conveys only density, so report the number.
Compare two graphs	Matrices with the same ordering	Node-link layouts of two graphs are not comparable at all (§1's non-determinism).

Several network questions are better answered by a non-network chart — the degree distribution, the centrality ranking, and the component sizes are all quantitative and **belong on position** (Module 02 §1). The don't-draw-the-graph option is underused and belongs in the table, which is why §4 develops it.

3 Hierarchies

- **A node-link tree** — depth encoded on position, which **makes the structure clearest of any option** and **uses space poorly for wide or deep trees**, since most of the canvas is empty.
- **An icicle plot or a sunburst** — depth on position and quantity on length (icicle) or angle (sunburst) — which **shows the structure and the quantity together**, and is the best general compromise. **Prefer the icicle, since length is rank three and angle is rank four.**
- **A treemap** — quantity on area, nesting on containment — which **uses space very efficiently and reads the quantity badly (rank five) and the structure worse still**, since deep nesting becomes invisible.
- **And an indented list**, which is a node-link tree with much better space efficiency and **no visual overview** — excellent for navigation and poor for seeing the shape.
- **So: a tree for structure, an icicle for structure and quantity together, and a treemap only when the space constraint genuinely dominates. A treemap puts quantity on area, which is rank five — so it is chosen for space efficiency and pays for it in reading accuracy, and that should be a deliberate trade rather than a default.**

4 When not to draw it

MOST NETWORK QUESTIONS ARE ABOUT A DERIVED QUANTITY, WHICH BELONGS ON A QUANTITATIVE CHART

The degree distribution, the centrality ranking, the connected component sizes, the clustering coefficient, and the path length distribution are all numbers (CSCE 676 Module 07's whole subject) — and **numbers go on position, which is rank one.**

So a degree distribution on log-log axes answers 'is this heavy-tailed' far better than any picture of the graph (CSCE 676 Module 07 §1), and **a sorted bar chart answers 'who is most central' better than any layout**, including one that sizes nodes by centrality.

Which means the network diagram is for the questions that are genuinely topological — 'how are these two connected', 'is there a path', 'what does this particular neighbourhood look like' — and those are a minority of the questions people bring to a graph.

And a hairball is a strong signal that the question was not topological in the first place — it is a figure that survived the choice of representation without ever passing through it, which is the diagnosis to make when you see one.

- **State the question, and check whether it is about the topology or about a derived quantity** — which takes a sentence and decides the rest.
- **If it is a quantity, plot the quantity** — which is **Module 03 §3's derivation argument applied to graphs**, and is the step most often skipped because the graph feels like the natural object.
- **If it is topological and the graph is small, use node-link** with a principled and deterministic layout (§1).

- If it is **topological** and the graph is large, use an **ordered matrix** (§2), or **extract the relevant subgraph and draw that** — a neighbourhood of fifty nodes is readable where the whole graph is not.
- And if none of those work, say that the graph is too large to show — which is an honest finding rather than a failure, and the alternative is a figure that implies a structure nobody can verify, which is Module 13's concern.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Munzner, chapter 9 (slides free)	§1 through §3 — node-link, matrix, and the hierarchy layouts, with the trades stated.
Ghoniem, Fekete & Castagliola — Node-link vs matrix (free)	§2's trade, measured — which tasks each representation wins, with the crossover point.
Behrisch et al. — Matrix reordering methods (free)	§2's ordering, surveyed — the methods, since the ordering is the whole design.
Shneiderman — Tree-maps (free)	§3's treemap in the original , with the space-filling motivation that justifies the area encoding.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Run a force layout twice** on the same graph and compare the pictures.
- 2 **Identify an apparent cluster** and check whether it survives a re-run.
- 3 **Draw a 500-node graph** as node-link and report what you can read.
- 4 **Draw the same graph as an alphabetically ordered matrix**, then reorder by a clustering.
- 5 **Trace a three-hop path** in each, and time it.
- 6 **Plot the degree distribution** and compare what it tells you against the node-link figure.
- 7 **Draw a hierarchy as a tree, an icicle, and a treemap**, and ask five people to compare two sizes.
- 8 **Find a published hairball** and state the question it was probably meant to answer.
- 9 **Answer that question with a non-network chart.**
- 10 **Extract a neighbourhood subgraph** and draw that instead.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does a force layout have no principled reading?
- 2 Why does non-determinism matter for a structural claim?
- 3 Give four things that make a node-link diagram readable.
- 4 State the matrix trade in both directions.
- 5 Why is the matrix ordering the whole design?
- 6 Give the representation for five network tasks.
- 7 Name four hierarchy layouts and what each preserves.
- 8 Why is a treemap a deliberate trade?
- 9 Which network questions belong on a quantitative chart?
- 10 What does a hairball signal?